Floodmonitor CheatSheet

Sebastian Nosek 4AHIT

Webclient

http://localhost:8080/index.html

Swagger UI

ttp://localhost:8080/swagger-ui/index.html

Open API

http://localhost:8080/v3/api-docs

Die 5 Endpunkte

1. ruft alle Station auf: **http://localhost:8080/api/v1/stations**

```
@GetMapping(value = "/stations", produces = {MediaType.APPL
ICATION_JSON_VALUE, MediaType.APPLICATION_XML_VALUE})
    public List<Station> getStations(@RequestParam(required
= false) String river,@RequestParam(required = false) Boole
an isActive, @RequestParam(required = false) WarningLevel w
arningLevel) {
        return simulationService.getfilterStationen(river,
isActive, warningLevel);
    }
```

2. ruft eine Station nur anhand der ID auf:http://localhost:8080/api/v1/stations/2

```
@GetMapping(value = "/stations/{id}", produces = {MediaTyp
e.APPLICATION_JSON_VALUE, MediaType.APPLICATION_XML_VALUE})
    public Station getStationById(@PathVariable String id)
{
        return simulationService.getStationById(id);
    }
```

3. Zeigt die letzten Messungen an:
   http://localhost:8080/api/v1/stations/measurements/latest

```
@GetMapping(value="/stations/{id}/measurements/latest", pro
duces = {MediaType.APPLICATION_JSON_VALUE, MediaType.APPLIC
ATION_XML_VALUE})
    public Measurement getLatesMeasurement(@PathVariable St
ring id) {
        return simulationService.getLatesMeasurement(id);
    }
```

4. zeigt nur die Messungen an, von einem bestimmten Zeitpunkt bis einem bestimmten Zeitpunkt: http://localhost:8080/api/v1/stations/measurements

```
@GetMapping(value = "/stations/{id}/measurements", produces
= {MediaType.APPLICATION_JSON_VALUE, MediaType.APPLICATION_
XML_VALUE})
    public List<Measurement> getMeasurements(@PathVariable
String id,@RequestParam(required = false) @DateTimeFormat(i
so = DateTimeFormat.ISO.DATE_TIME) LocalDateTime from,@Requ
estParam(required = false) @DateTimeFormat(iso = DateTimeFo
rmat.ISO.DATE_TIME) LocalDateTime to,@RequestParam(required
= false) WarningLevel warningLevel, @RequestParam(required
= false, defaultValue = "100") Integer limit) {
    if(limit < 1 || limit > 500) {
        throw new IllegalArgumentException("Das Limit darf
nur zwischen 1 und 500 sein");
    }
    if(from != null && to != null && from.isAfter(to)) {
        throw new IllegalArgumentException("From muss vor T
o sein");
    }
    return simulationService.getMeasurements(id, from, to,
warningLevel, limit);
    }
```

5. Zeigt die alerts an, also Warning und Crtitical:
   http://localhost:8080/api/v1/alerts

```
@GetMapping(value = "/alerts", produces = {MediaType.APPLIC
ATION_JSON_VALUE, MediaType.APPLICATION_XML_VALUE})
    public List<Measurement> getAlerts(@RequestParam(requir
ed = false) WarningLevel warningLevel, @RequestParam(requir
ed = false) String river) {
        return simulationService.getAlerts(warningLevel, ri
ver);
    }
```

Warnstufen

Station	Warnwert	Kritisch	Start	Warnstufe
Wien	320	430	250	Normal
Linz	310	420	330	Warning
Köln	420	500	520	Critical
Krakow	330	440	-	Unknown

In der Methode calculateWarningLevel aus der Klasse SimulationService, rechnet bzw filter es für jede Station die eigene Warnstufe, mithilfe von dem WaterLevel, CriticalWaterLevel und WarningWaterLevel.

Automatisierte Tests

@MockBean ersetzt den echten Service durch einen Fake, die Zufallssimulation ist sozusagen abgeschalten, damit die Tests nicht zufällig fehlschlagen

Muster: `when(...).thenReturn(...)` → Wenn der Controller fragt, antworte so, `thenThrow` = Fehler simulieren

Beispiel → testet den Abruf aller Stationen

```
@Test
    public void test1_GetStationsJson() throws Exception {
        when(simulationService.getfilterStationen(any(), an
y(), any())) //any(), brauche ich, wenn der Filter nach etw
as fragt, damit der Wert egal ist.
                .thenReturn(Collections.singletonList(testS
tation("1")));       //es antwortet mit einer singletonList
```

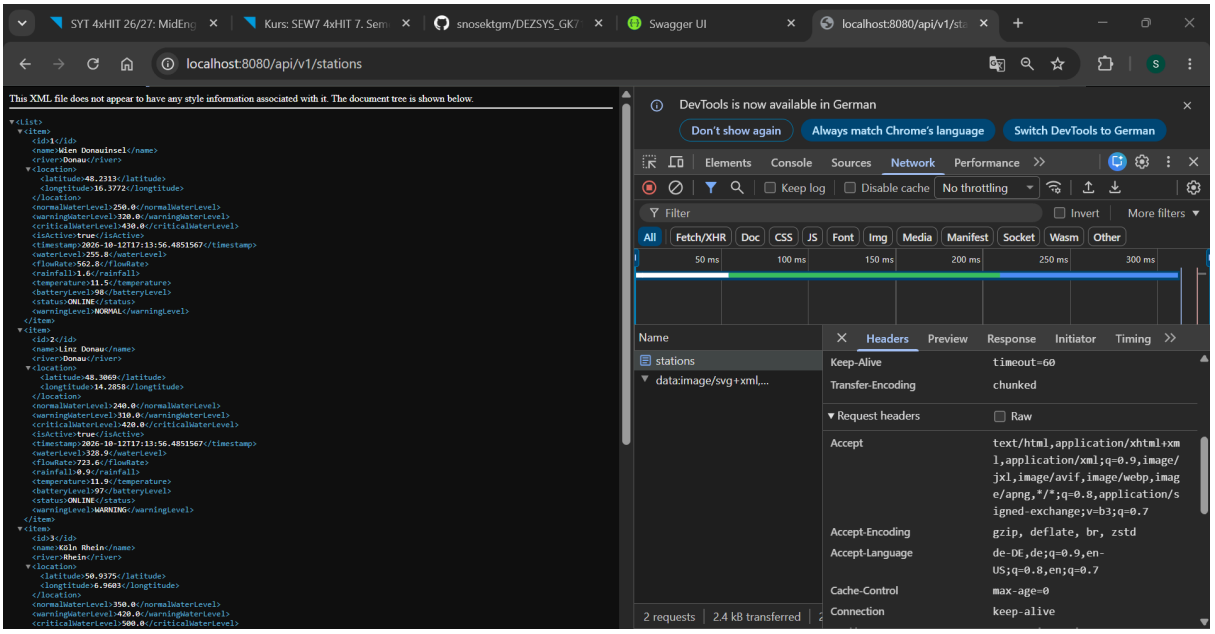
```
e, das heißt eine Liste mit nur einem Eintrag

        mockMvc.perform(get("/api/v1/stations").accept(Medi
aType.APPLICATION_JSON))     //hier wird mit der GET api/v
1/stations gemockt
                .andExpect(status().isOk())
                .andExpect(content().contentType(MediaType.
APPLICATION_JSON))
                .andExpect(jsonPath("$[0].id").value("1"));
    }
```

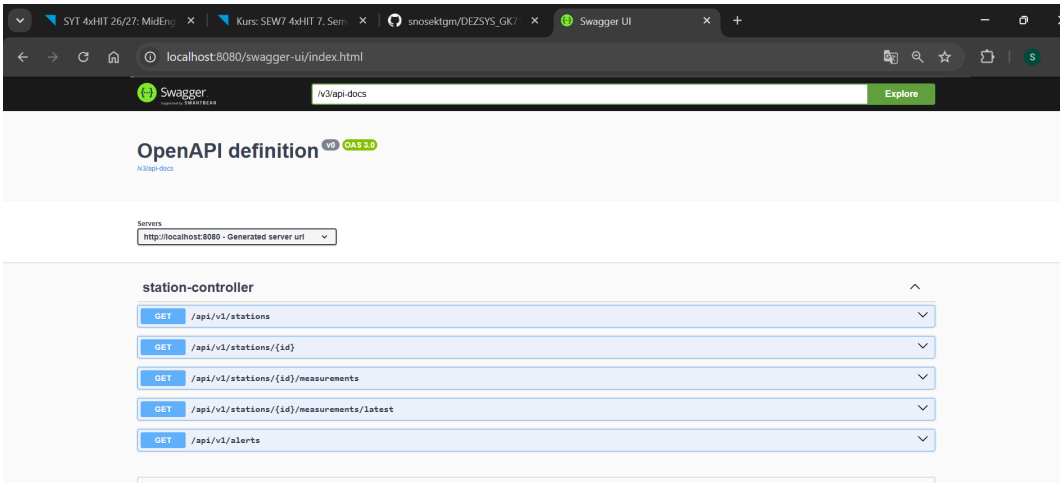
Content Negotiation

```
produces = {MediaType.APPLICATION_JSON_VALUE, MediaType.APP
LICATION_XML_VALUE}
```

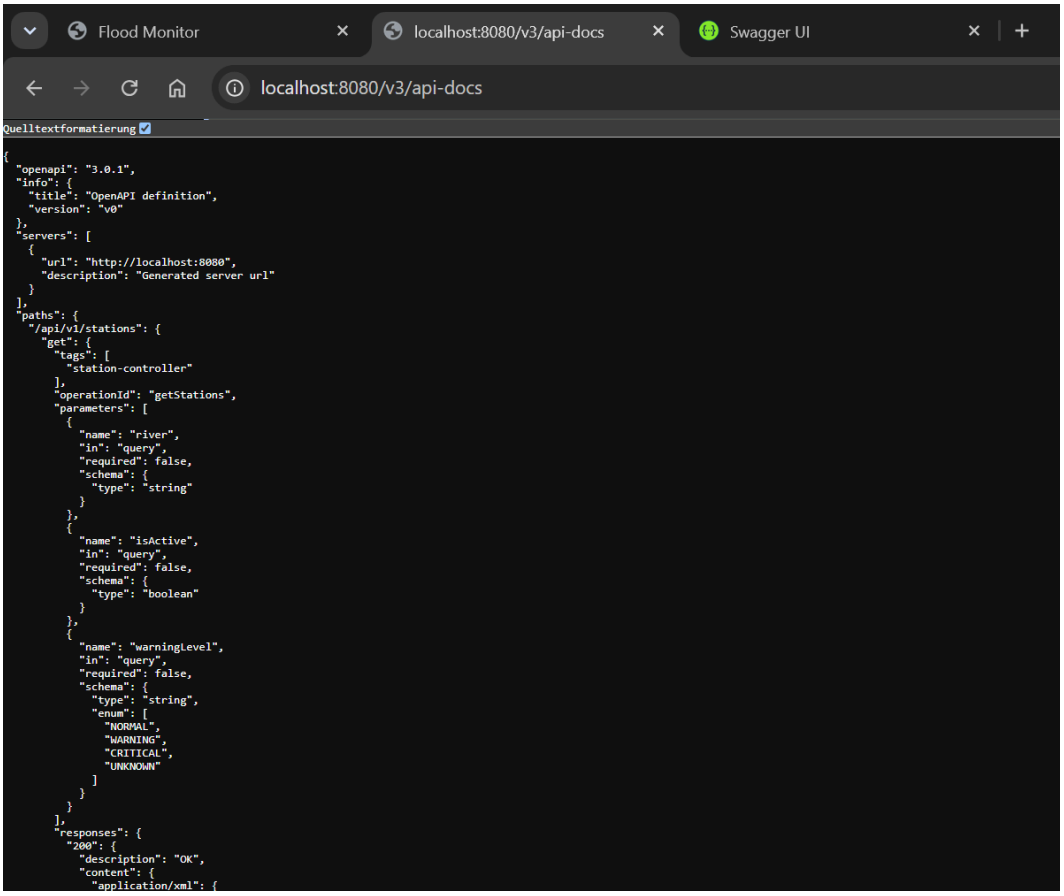
Das ist im Controller drinnen, die URL fragt normalerweise nach einer html, wenn es keine html gibt, fragt es nach einer xml, das kann man bei Network im Accept-Header sehen:



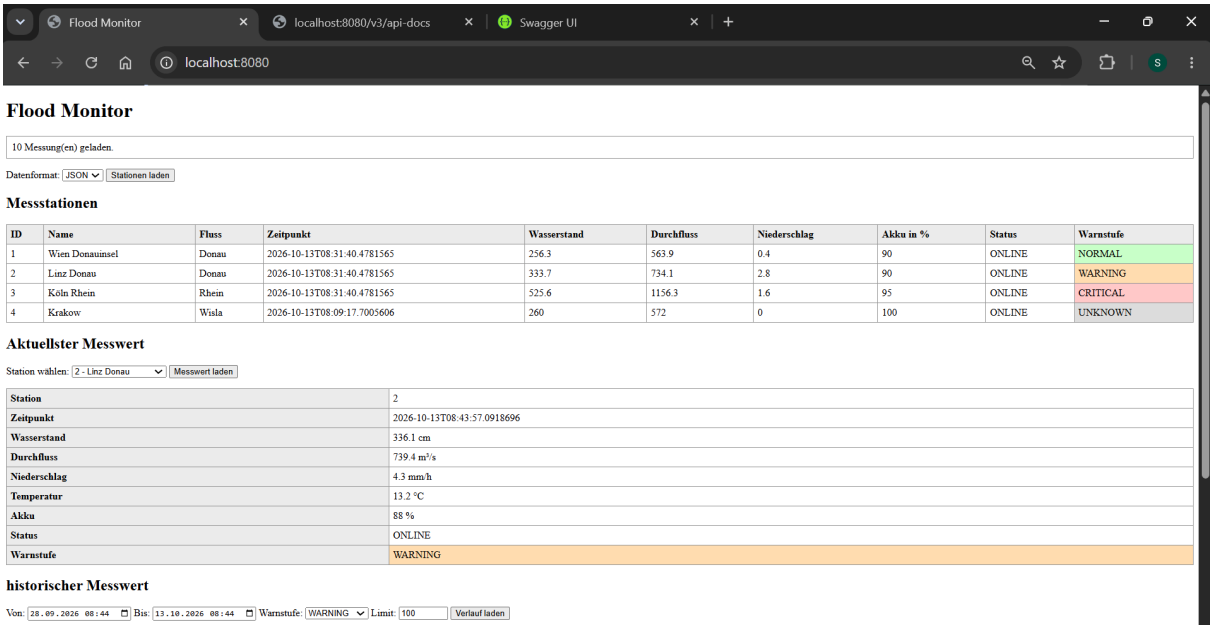
Swagger UI



OpenAPI



Web-Interface



Wichtige Annotations

- @RestController → markiert die Klasse als den REST-API Endpunkt
- @RequestMapping → definiert den Basispfad für alle Endpunkte im Controller Bsp. @RequestMapping(”/api/v1”)
- @GetMapping → verknüpft eine Methode mit einer HTTP-GET Anfrage Bsp. @GetMapping(value=”/stations”)
- @PathVariable → liest Variablen direkt aus dem URL-Pfad ein Bsp. @PathVariable String id
- @RequestParam → liest URL-Parameter aus, wie ein Filter. Kann aber auch Optional sein. Bsp. @RequestParam(required = false) String river
- @RequestHeader → greift auf HTTP-Header zu, beispielsweise um den Accept-Header für die XML Überprüfung

Aufgabe der Klassen

- StationController → Das ist die Klasse mit den REST-Endpunkten, also mit den @GetMapping Methoden.

- SimulationService → Da findet die ganze Logik statt, mit der simulation, dem berechnen der Warninglevel und mit den Filtern.
- FloodMonitorRepository → das ist ein Speicher für die ganzen Messungen.
- GlobalExceptionHandler → Das ist die Exception Klasse für alle fehler (IllegalArgumentException, ResponseStatusException)
- Station / Location / Measurement → Das sind Klassen nur mit Daten (Getter - Setter Methoden)
- WarningLevel/StationStatus → Sind Enum-Klassen
- index.html → Ist mein Webclient, er sendet das Format per Dropdown
- StationControllerTest → Hier mocke ich den Service, wodurch die Simulation nicht mehr funktioniert und die Tests nicht plötzlich fehlschlagen können.

Wichtige HTTP-Statuscodes

- status().isOk() → HTTP200 (Alles erfolgreich)
- status().isNotFound() → HTTP404 (Etwas nicht gefunden)
- status().isBadRequest() → HTTP400 (ungültiger Parameter)
- status().isNotAcceptable() → 406 (Format wird vom Server nicht unterstützt)

Gradle Befehle

Projekt teste → .\gradlew test

Projekt starten → .\gradlew bootRun

Build aufräume → .\gradlew build/clean